PROCESSO SELETIVO CITI — DESAFIO DE BACK-END

Relatório Técnico: API Folhagem & Cia

Uma nota antes de começar: usei IA durante o projeto e também para organizar este relatório. O Claude me ajudou a montar a estrutura e a revisar o texto; o conteúdo, porém, conta o que de fato aconteceu no meu desenvolvimento. Detalho tudo na seção 4.

1. O que foi feito

Construí a API da Folhagem & Cia, que cuida do estoque de plantas e do registro de vendas. Implementei as 7 rotas do enunciado:

- `POST /plantas`: cadastro de planta;
- `GET /plantas/:id`: busca por ID;
- `PATCH /plantas/:id`: edição parcial;
- `GET /plantas`: listagem, com filtros de luminosidade e petFriendly;
- `POST /plantas/:id/venda`: registro de venda;
- `GET /plantas/estoque/valor-total`: valor total do estoque;
- `DELETE /plantas/:id`: remoção de planta.

Nas validações, tentei cobrir o que costuma escapar: a API recusa tipos errados (texto onde deveria ser número ou boolean, decimal onde deveria ser inteiro), números acima do limite de 32 bits (`2147483647`), preço zerado ou negativo, estoque negativo e requisições sem corpo (respondem 400 Bad Request). Também protegi a venda para que duas compras ao mesmo tempo não passem do estoque disponível.

Não deixei nenhuma rota ou regra da especificação de fora. Também não fiz extras além do enunciado: preferi gastar o tempo deixando essas 7 rotas bem testadas e estáveis.

2. Como foi feito

Fui nesta ordem: schema e migration, repository, controller e rotas; depois validação, testes e limpeza. Algumas decisões que vale contar:

- **Camadas separadas.** Deixei o acesso ao banco no `PlantaRepository` e as rotas e respostas HTTP no `PlantaController`, seguindo o padrão do `UserController` que já vinha no projeto.
- **Validar antes de ir ao banco.** No cadastro e no PATCH, checo o payload primeiro. Se estiver inválido, a API responde 400 na hora, sem gastar uma consulta à toa.
- **Venda atômica.** Para duas compras simultâneas não deixarem o estoque negativo, usei `updateMany` com a condição `estoque: { gte: quantidade }`. Assim a checagem e o desconto acontecem num único comando no banco.
- **Lockfiles.** Usei npm por costume e isso gerou um `package-lock.json`. No fim, restaurei o `yarn.lock` original da main e deixei o `package-lock.json` fora do Git, via `.git/info/exclude`.
- **Migrations.** Apliquei no container com `docker compose exec server npx prisma migrate deploy`.
- **Testes.** O `testes.sh` reúne testes de caixa-preta cobrindo os cenários do enunciado e casos de borda: são 62 verificações, todas passando. O único teste que fiz à mão foi conferir se a rota de valor total devolve zeros quando a loja está vazia.

3. As principais dificuldades

1. **Porta 5432 ocupada.** O container não subia porque eu já tinha um PostgreSQL rodando na máquina. Mudei a porta no `.env` (que não vai para o Git) para `5433`.
2. **findAllFiltered não existia.** O `PlantaController` chamava esse método, mas na revisão com o Claude percebi que ele nem existia no `PlantaRepository`. Implementei o filtro por luminosidade e petFriendly.
3. **Preço zero passando.** No começo eu validava `preco < 0`, então era possível cadastrar planta com preço 0. Notei na revisão e troquei para `preco <= 0`.
4. **IDs gigantes derrubando o banco.** IDs acima de 2.147.483.647 causavam erro 500 no Prisma. Criei o `parseIdParam` para barrar esses valores e responder 400.
5. **Estoque negativo em compras simultâneas.** Minha lógica inicial lia o estoque e só depois atualizava, o que abre espaço para condição de corrida. Com uma dica do Claude, passei para a atualização atômica com `updateMany`.
6. **package-lock.json no Git.** Commitei o arquivo sem querer no commit `453566b`. Depois removi da árvore e restaurei o `yarn.lock` no commit `82a484c`. O histórico intermediário guarda a alteração, mas a branch final está limpa.
7. **Funções sem uso.** No fim, `validarPlanta` e `isValidEstoque` (em `validators.ts`) ficaram sem uso, então apaguei as duas.

4. O uso de IA

Usei duas ferramentas de IA, cada uma de um jeito:

- **Claude (Anthropic):** foi meu assistente principal durante o projeto e também na organização deste relatório.
- **Gemini (Google):** usei de forma pontual, como apoio em alguns trechos de código.

O que eu fiz. Escrevi a maior parte do código das rotas, do `PlantaController`, do `PlantaRepository` e do `validators.ts`, além do schema do Prisma, e estudei por conta própria a estrutura do `UserController` que serviu de base.

O que a IA fez. O Claude escreveu o script de testes de caixa-preta (`testes.sh`), sugeriu a operação atômica (condição e decremento na mesma query) para resolver a concorrência na venda e revisou meu código, alertando sobre o `findAllFiltered` ausente, o `preco <= 0` e o limite de 32 bits nos IDs. Ele também me ajudou a resolver erros no código que escrevi e a estruturar este relatório.

Como conferi. Não aceitei nada sem validar, venha de qual ferramenta vier. Rodei o script de testes (62 verificações, todas passando), conferi os tipos com `tsc --noEmit` e refiz o build do zero (`docker compose down -v`).

O que aprendi. Aprendi a tratar concorrência no banco sem race condition, a validar limites numéricos com rigor em APIs Node/TypeScript e a lidar melhor com lockfiles no Git.